

z/Architecture concepts

The plan

- In this unit, we'll present the main components of a z/Architecture computer that Assembler programs interact with:
 - Memory
 - Central Processing Unit (CPU, aka “the computer”)
 - Program Status Word (PSW)
 - Registers
 - Instructions
- ...starting with Memory, next.

Memory - bytes

- z/Architecture memory (also called Main Storage) is organized in 8-bit units called **Bytes**
- Data and instructions are stored in bytes in memory
- Numbers of bytes are expressed in units like Kilobytes (KB), Megabytes (MB)...
 - You will have heard that (say) a laptop has a certain number of Gigabytes
 - Next, a summary of these units, for reference

Counting bytes

- Below are power-of-2 units widely used in computing
 - We will mention them throughout the course
 - For example: we'll refer to 1,048,576 bytes as 1 Megabyte (1 MB)

	Value	Unit name	Short
2^{10}	1024	Kilobyte	KB
2^{20}	1048576	Megabyte	MB
2^{30}	1073741824	Gigabyte	GB
2^{40}	1099511627776	Terabyte	TB
2^{50}	1125899906842620	Petabyte	PB
2^{60}	1152921504606850000	Exabyte	EB

Counting bytes

- A few values we'll come across in the course, as they are frequently used in z/Architecture

	Value	Bytes
2^{15}	32768	32 KB
2^{16}	65536	64 KB
2^{24}	16777216	16 MB
2^{31}	2147483648	2 GB
2^{32}	4294967296	4 GB
2^{63}	9223372036854780000	8 EB
2^{64}	18446744073709600000	16 EB

Memory - bits in bytes

- Bits in a byte are numbered 0 to 7, left (most significant) to right (least)
 - Most significant bits (left) are called “high-order”
 - Least significant (right) are called “low-order”

Bit number	0	1	2	3	4	5	6	7
Contents (example)	1	1	1	1	0	0	0	1

Bytes as characters

- A byte can hold 256 different values, from (hexadecimal) **00** to **FF**
- By convention, certain byte values represent characters, for example '**A**', '**W**', '**5**', '*****', '**&**' ...
 - When a program prints or displays a byte, it prints or displays the character it represents
- z/Architecture uses, by default, the EBCDIC (Extended Binary Coded Decimal Interchange Code) standard to represent characters
 - z/Architecture also supports ASCII (American Standard Code for Information Interchange)

Bytes as characters - EBCDIC

- Alphanumeric EBCDIC character representations:

Hexadecimal	Character
F0 to F9	Numbers 0 to 9
C1 to C9	Uppercase letters A to I
D1 to D9	Uppercase letters J to R
E2 to E9	Uppercase letters S to Z
81 to 89	Lowercase letters a to i
91 to 99	Lowercase letters j to r
A2 to A9	Lowercase letters s to z

- Example: in an EBCDIC computer, the characters **ABC** are stored in memory as **C1C2C3**

Bytes as characters - EBCDIC

- A few more examples of EBCDIC character representations:

Hexadecimal	Character	Hexadecimal	Character
40	Space (" ")	5C	Asterisk ("*")
4B	Period (".")	60	Minus sign ("-")
4E	Plus sign ("+")	61	Forward slash ("/")
50	Ampersand ("&")	6B	Comma (",")
5B	Dollar sign ("\$")	7D	Single quote ("'")

- Example: in an EBCDIC computer, the characters **/*** are stored in memory as hex **615C**

Bytes as characters - ASCII

- Note: ASCII presented for completeness; most of the course will deal with EBCDIC
- Alphanumeric ASCII character representations:

Hexadecimal	Character
30 to 39	Numbers 0 to 9
41 to 5A	Uppercase letters A to Z
61 to 7A	Lowercase letters a to z

- Example: in an ASCII computer, the characters **ABC** are stored in memory as **414243**

Bytes as characters - ASCII

- A few more examples of ASCII character representations:

Hexadecimal Character		Hexadecimal Character	
20	Space (" ")	2D	Minus sign ("-")
24	Dollar sign ("\$")	2E	Period (".")
26	Ampersand ("&")	2F	Forward slash ("/")
2A	Asterisk ("*")	5C	Backslash ("\")
2B	Plus sign ("+")	60	Backtick ("`")
2C	Comma (",")		

A byte's “meaning”

- The “meaning” of a byte depends on how it is used
- Example 1: two bytes containing hexadecimal **605C** can be interpreted as:

Halfword binary integer	+24,668	Value in a binary calculation
EBCDIC characters	"-*"	Displayed or printed on an EBCDIC device
ASCII characters	"`\"	Displayed or printed on an ASCII device
Packed decimal (*)	+605	Value in a decimal calculation
Even an instruction! (*)	60	Store from a floating-point register to memory

(*) We'll introduce packed decimal and instructions later

A byte's “meaning”

- Example 2: given two bytes containing hexadecimal **F1F0**:
- When displayed or printed on an EBCDIC device, they will appear as ‘**10**’
- If used as a binary halfword, their value is decimal **-3600**
 - So, if we wanted to use them as decimal **10** in a calculation, we must convert them to a binary integer (for example, halfword ‘**000A**’) first
 - We’ll learn how to do this later

Addressing bytes

- Bytes in memory are arranged so that each byte is individually numbered **beginning at zero**
- The number associated with each byte is called its **memory address**
- Memory may be thought of as a string of bytes
- Note *only bytes have addresses*; bits within a byte don't have their own addresses

Addressing bytes

- Memory addresses are usually expressed in hexadecimal:

[illegible]

Groups of bytes

- Many machine instructions operate on groups of bytes:

Group size (bytes)	Group name
2	Halfword
4	Fullword
8	Doubleword
16	Quadword

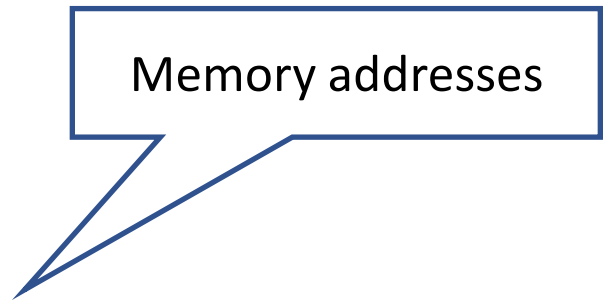
- The group's memory address is that of the leftmost byte

Groups of bytes

- Some instructions require that the group address is a multiple of its size in bytes
 - For example, a fullword at a multiple-of-four address
- In this case, the group is said to be **aligned**

Groups of bytes

- Example of aligned groups:



8A0	8A1	8A2	8A3	8A4	8A5	8A6	8A7	8A8	8A9	8AA	8AB	8AC	8AD	8AE	8AF
halfword		halfword		halfword		halfword		halfword		halfword		halfword		halfword	
fullword				fullword				fullword				fullword			
doubleword								doubleword							
quadword															

Data and programs

- In addition to data, a z/Architecture computer memory also stores **programs**
 - Most modern computers are like this
 - This is called Von Neuman architecture
- We have yet another interpretation for data in memory: not just binary, character, or decimal, but also **instructions**
- How does a z/Architecture computer distinguish between data and instructions? Time to introduce the CPU!